

Foundations of Object-Oriented Programming: A Mathematical and Applied Analysis

Abstract—This paper bridges the theoretical mathematical abstractions of Object-Oriented Programming (OOP) paradigms with practical implementation mechanics. By transitioning from descriptive heuristics to rigorous mathematical formulations, we explore classes, objects, memory management, object duplication, and Linked Lists using set theory, mappings, and corresponding structural code examples.



1 Introduction

The adoption of Object-Oriented Programming (OOP) requires a paradigm shift towards encapsulated states and behaviors. Formally, OOP models a software system as a set of interacting discrete entities. Unlike procedural paradigms modeled as a sequence of functions $\mathcal{F}$ applied to a global state space S , OOP defines isolated, partitioned state spaces S_i mapped to specific behavioral operations M_i .

2 Core Concepts: Classes and Objects

We formally define the pillars of OOP utilizing set theory.

- Class ($\mathcal{C}$): A class is an ordered pair $\mathcal{C} = (A, M)$, where A represents a finite set of attributes and M represents a finite set of methods.
- Object (o): An instantiated entity $o \in O$. The instantiation is a surjective mapping $I : \mathcal{C} \rightarrow O$.

Practical Implementation:

```
class Point {
  int x; // a_1 in A
  int y; // a_2 in A

  // m_1 in M
  void move(int dx, int dy) {
    this.x += dx;
    this.y += dy;
  }
}
// Instantiation: I(Point) -> o
Point p = new Point();
```

3 Memory Management: Stack vs. Heap Spaces

Memory architecture is modeled as disjoint sets: the Stack (S) and the Heap (H). A reference variable $r \in S$ acts as a pointer mapping, defined by $f : S \rightarrow H \cup \{\emptyset\}$, where $f(r) = h$.

Practical Implementation:

```
Point p1 = new Point();
// 'new Point()' allocates contiguous block h in H.
// 'p1' is a reference r in S. f(p1) = h.

Point p2 = null;
// 'p2' is reference r in S. f(p2) = empty set.
```

4 Object Duplication: Shallow vs. Deep Copy

Let o_1 reside at heap address $h_1 \in H$, referenced by $r_1 \in S$ such that $f(r_1) = h_1$.

- Shallow Copy: Maps a new reference $r_2 \in S$ to the exact same heap address: $f(r_2) = f(r_1) = h_1$.
- Deep Copy: Allocates a new heap address $h_2 \in H$ ($h_1 \neq h_2$) for object o_2 , followed by element-wise state cloning.

Practical Implementation:

```
// Shallow Copy
Point p_shallow = p1;
// f(p_shallow) = f(p1) = h_1

// Deep Copy (Manual element-wise cloning)
Point p_deep = new Point();
p_deep.x = p1.x;
p_deep.y = p1.y;
// f(p_deep) = h_2 (disjoint from h_1)
```

5 Memory Optimization: Static Data Members

To optimize space complexity, a static member $a_s \in A_{static}$ is bound to the class definition $\mathcal{C}$ itself. This reduces the space complexity for N objects from $\mathcal{O}(N)$ to $\mathcal{O}(1)$.

Practical Implementation:

```
class Student {
  String name; // Instance variable, O(N) space
  static String university = "MIT"; // Static, O(1) space
}
```

6 Linear Data Structures: Linked Lists

A Linked List L is a directed acyclic graph (DAG). A node N_i is a tuple (d_i, p_i) , where d_i is data and $p_i : N_i \rightarrow N_{i+1}$ is a directed edge mapping.

6.1 Traversal and Operations

Traversal to the K -th node requires applying the mapping function K times:

$$N_k = p^{(K)}(N_0) = \underbrace{p \circ p \circ \dots \circ p}_{K \text{ times}}(N_0)$$

Practical Implementation:

```

class Node {
    int data; // d_i
    Node next; // p_i
}

Node head = new Node(); // N_0
Node temp = head;

//  $p^K(N_0)$  iterative mapping
for (int i = 0; i < K; i++) {
    if (temp != null) {
        temp = temp.next; // temp  $\leftarrow$  p(temp)
    }
}

```

7 Conclusion

The theoretical foundations of Object-Oriented Programming, when paired with their structural implementations, provide a robust, deterministic environment. Modeling paradigms with rigorous mapping functions and validating them through execution semantics forms a complete perspective on modern computational architectures.